

实验6报告：基于 SegNet 的 CamVid 语义分割

1. 实验目的

本实验旨在实现并验证一个基于 SegNet 的街景语义分割系统，在 CamVid 数据集上完成像素级分类任务，并重点分析：

- 主干类别（Sky、Road、Building）与小目标类别（Pole、SignSymbol、Pedestrian、Bicyclist）的性能差异。
- 训练稳定性与泛化能力。
- 类别不均衡和边界混淆对 mIoU 的影响。

2. 实验环境

2.1 软件环境

- 操作系统：Windows
- Python：3.11.15
- 深度学习框架：PyTorch 2.11.0+cu130
- CUDA：13.0（可选）
- 可视化依赖：matplotlib

2.2 关键依赖

- numpy >= 1.24
- Pillow >= 10.0
- torch >= 2.1

依赖文件位于：[src_to_submit/exp6/requirements.txt](#)。

3. 数据集与预处理

3.1 数据集说明

本实验使用 CamVid 11 类语义分割数据，目录结构为：

- [train/](#) 与 [train_labels/](#)
- [val/](#) 与 [val_labels/](#)
- [test/](#) 与 [test_labels/](#)

本次运行环境中的样本规模统计为：

- 训练集：367
- 验证集：101
- 测试集：233

3.2 预处理与增强

本实验采用以下图像处理策略：

- 输入尺寸统一到 360 x 480。
- 训练阶段使用随机缩放裁剪 (scale: 0.75 到 1.25)。
- 训练阶段使用随机水平翻转。
- 训练阶段使用亮度与对比度扰动 (0.15)。
- 使用 ImageNet 均值方差归一化。

标签处理方面，支持灰度类别图和 RGB 颜色标注图，并在载入阶段映射为类别索引。

3.3 数据管线核心实现

与实验4类似，这里通过实现示例说明数据是如何从目录组织到可训练张量的。

1. 数据划分读取：按 train/val/test 三组目录自动发现样本。

```
def load_camvid_splits(data_dir: Path) -> dict[str, list[SegmentationSample]]:
    split_pairs = {
        "train": (data_dir / Config.train_image_dir, data_dir /
Config.train_mask_dir),
        "val": (data_dir / Config.val_image_dir, data_dir / Config.val_mask_dir),
        "test": (data_dir / Config.test_image_dir, data_dir /
Config.test_mask_dir),
    }
    return {
        split: discover_split_samples(image_dir=image_dir, mask_dir=mask_dir)
        for split, (image_dir, mask_dir) in split_pairs.items()
    }
```

2. 标签文件名容错：除了同名，还支持 _L / _label 后缀匹配。

```
def _resolve_mask_name_candidates(image_path: Path) -> list[str]:
    stem = image_path.stem
    suffix = image_path.suffix
    return [
        f"{stem}{suffix}",
        f"{stem}.png",
        f"{stem}_L{suffix}",
        f"{stem}_L.png",
        f"{stem}_label{suffix}",
        f"{stem}_label.png",
    ]
```

3. 训练增强：随机缩放裁剪 + 翻转 + 亮度对比度扰动。

```
image, mask = self._random_rescale_and_crop(image, mask)
image, mask = self._maybe_flip(image, mask)
image = self._maybe_color_jitter(image)
```

4. 类别重加权：从训练标签统计频率后生成权重。

```
counts = np.maximum(counts, 1.0)
freq = counts / counts.sum()
weights = np.power(1.0 / freq, float(power))
weights = weights / weights.mean()
return torch.tensor(weights, dtype=torch.float32)
```

4. 模型与训练配置

4.1 模型结构

本实验采用 SegNet 编码器-解码器结构，核心特点如下：

- 编码端通过多层 Conv-BN-ReLU 提取语义特征。
- 使用 MaxPool(return_indices=True) 记录池化索引。
- 解码端使用 MaxUnpool 按索引恢复空间结构。
- 最后用 1x1 卷积输出 11 类 logits。

当前通道配置：

- encoder_channels = (64, 128, 256, 512, 512)
- block_depths = (2, 2, 3, 3, 3)

4.2 训练配置

默认训练配置（见 config.py）：

- batch_size = 2
- epochs = 100
- lr = 1e-3
- weight_decay = 1e-4
- optimizer = Adam
- scheduler = cosine
- min_lr_ratio = 0.1
- grad_clip = 1.0
- use_amp = True

损失函数设置：

- CrossEntropy 为主（ce_loss_weight = 1.0）
- 默认不开启 Dice（dice_loss_weight = 0.0）
- 启用类别加权（use_class_weights = True）以缓解长尾类别不平衡

4.3 训练与推理流程

1. 命令行模式与实验入口：

```

parser.add_argument("--mode", choices=["train", "eval", "predict"], required=True)
...
if args.mode == "train":
    run_train(args)
elif args.mode == "eval":
    run_eval(args)
else:
    run_predict(args)

```

2. 训练主循环：每个 epoch 完成 train -> val/test evaluate -> 保存 best/last。

```

for epoch in range(start_epoch, args.epochs + 1):
    train_loss = train_one_epoch(...)
    val_stats = evaluate(...)
    test_stats = evaluate(...)
    if scheduler is not None:
        scheduler.step()
    if val_miou >= best_miou:
        save_checkpoint(..., ckpt_path=best_ckpt_path, ...)
    save_checkpoint(..., ckpt_path=last_ckpt_path, ...)

```

3. AMP + 梯度裁剪细节：

```

with torch.autocast(device_type="cuda", dtype=torch.float16, enabled=use_amp):
    logits = model(images)
    loss = criterion(logits, masks)

if use_amp and scaler is not None:
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
    scaler.step(optimizer)
    scaler.update()

```

4. 指标统计逻辑：混淆矩阵累积并统一计算 PA/MPA/mIoU/fwIoU。

```

labels = self.num_classes * targets_np[mask] + preds_np[mask]
counts = np.bincount(labels, minlength=self.num_classes ** 2)
self.confusion_matrix += counts.reshape(self.num_classes, self.num_classes)
...
mean_iou = float(iou[union > 0].mean()) if np.any(union > 0) else 0.0

```

5. 评估指标

设混淆矩阵为 M ，类别数为 C ，采用如下指标：

- $PA = \frac{\sum_i M_{ii}}{\sum_{i,j} M_{ij}}$
- $MPA = \frac{1}{C} \sum_i \frac{M_{ii}}{\sum_j M_{ij}}$
- $IoU_i = \frac{M_{ii}}{\sum_j M_{ij} + \sum_j M_{ji} - M_{ii}}$
- $mIoU = \frac{1}{C} \sum_i IoU_i$
- $fwIoU = \frac{1}{\sum_{i,j} M_{ij}} \sum_i \left(\sum_j M_{ij} \right) IoU_i$

其中 mIoU 为本实验主要模型选择指标。

6. 实验结果与分析

6.1 本次过测设置

为快速验证提交流程是否可运行，本次采用“轻量过测”方案：

- 直接复用已有权重：`src/exp6/checkpoints/segnet_camvid.best.pth`
- 执行 eval 与 predict，全流程在提交代码目录 `src_to_submit/exp6` 下完成
- 预测输出目录：`src_to_submit/exp6/visualizations/predictions_smoke`

该目录共生成预测图 233 张，与测试集样本数量一致。

6.2 整体指标

本次实测输出如下：

数据集	Loss	PA	MPA	mIoU	fwIoU
Val	0.2301	0.9246	0.7511	0.6261	0.8764
Test	0.3746	0.8784	0.6580	0.5177	0.8073

可以看到：

- 验证集指标整体高于测试集，说明跨场景泛化仍有下降。
- PA 与 fwIoU 较高，但 mIoU 相对较低，说明类别不均衡下主类主导明显。

6.3 测试集类别 IoU 结果

类别	IoU
Sky	0.9071
Building	0.7449
Pole	0.2120
Road	0.9733
Pavement	0.0000
Tree	0.6655
SignSymbol	0.2205

类别	IoU
Fence	0.2178
Car	0.6994
Pedestrian	0.2827
Bicyclist	0.2534

类别层面的主要现象：

- 强势类别：Road、Sky、Building、Car。
- 薄目标或小目标类别：Pole、SignSymbol、Fence、Pedestrian、Bicyclist 表现明显偏低。
- Pavement IoU 为 0，提示 Road 与 Pavement 边界或标注分布存在显著混淆。

6.4 结果解读

1. 模型对大面积背景类学习充分。
2. 小目标与细长结构对下采样过程敏感，信息易丢失。
3. 单尺度输入和基础 SegNet 容量对复杂边界场景仍不足。
4. 仅使用 CE + 类别加权在极难类上的提升有限，后续需要更强损失或结构改进。

6.5 样例层面分析

结合预测输出目录中的样例图可观察到以下规律：

1. 对于天空、道路、建筑等大面积连续区域，预测掩码整体平滑且轮廓连贯。
2. 对于杆件、交通标志、行人、自行车等小目标，容易出现漏检或被邻近大类吞并。
3. 在道路边缘区域，Pavement 常被预测为 Road，和类别 IoU 中 Pavement=0.0000 的现象一致。

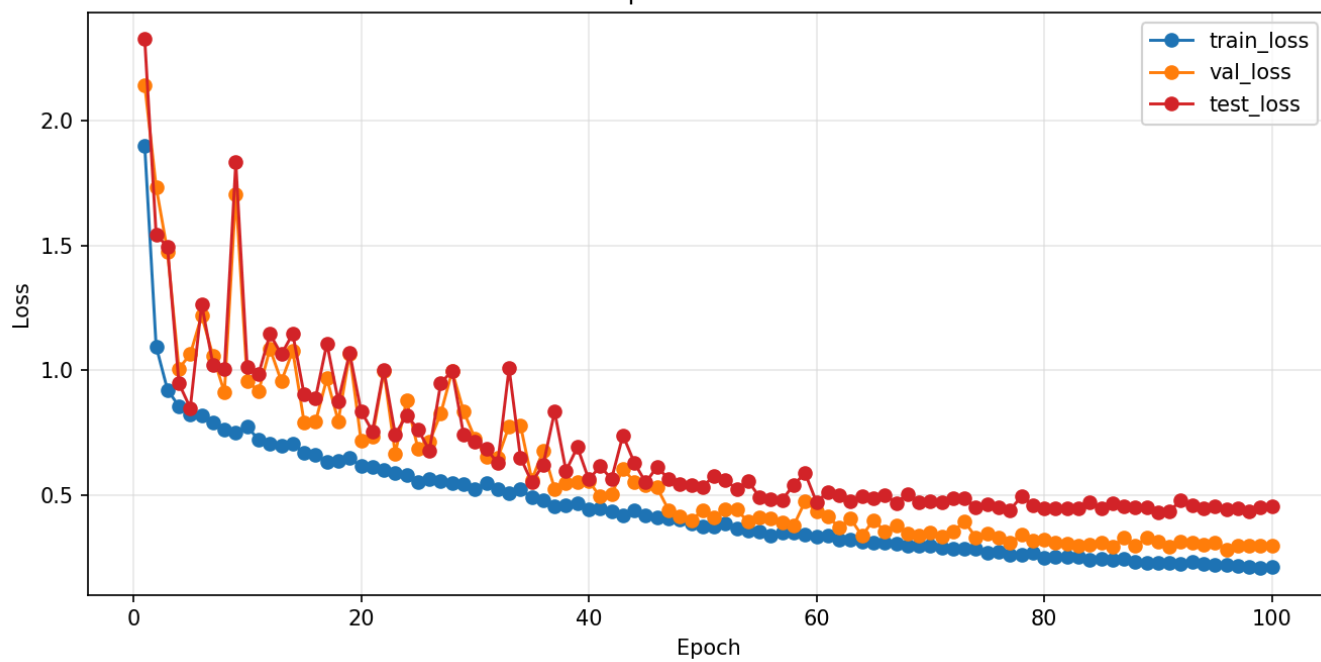
这种“主类强、小类弱”的结构性现象，与 CamVid 类别分布不均衡及 SegNet 下采样信息损失机制是吻合的。

7. 可视化与产物

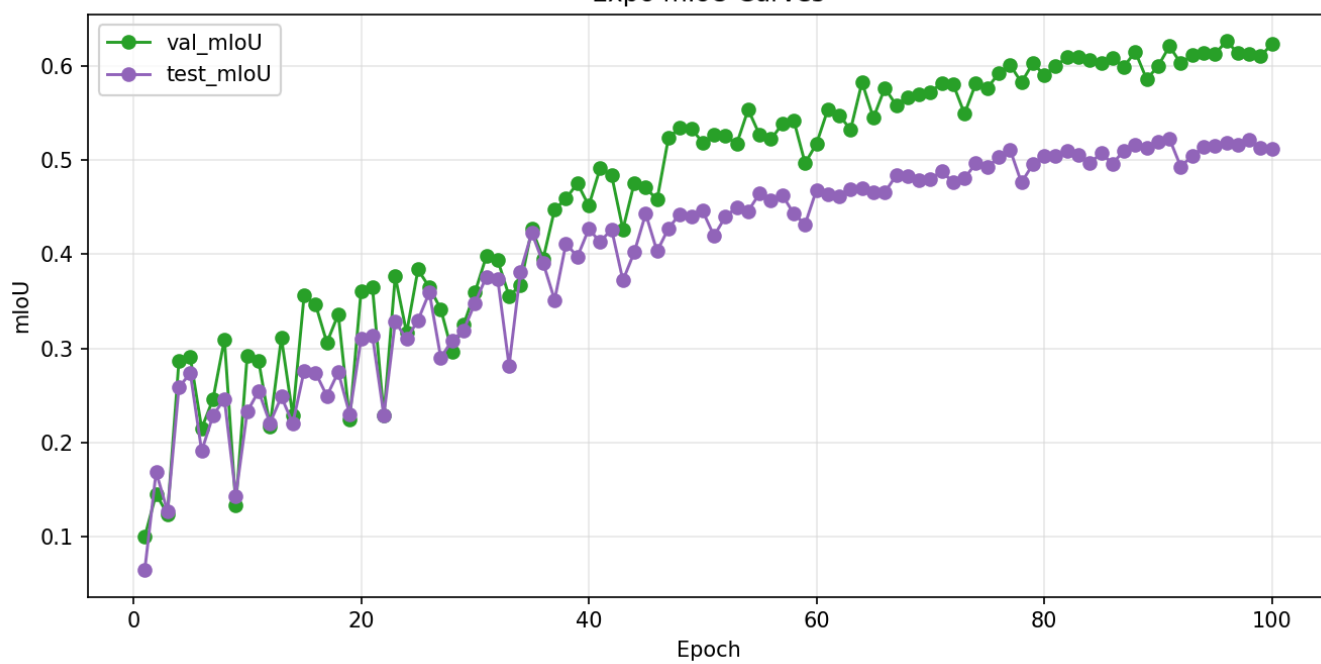
7.1 训练曲线

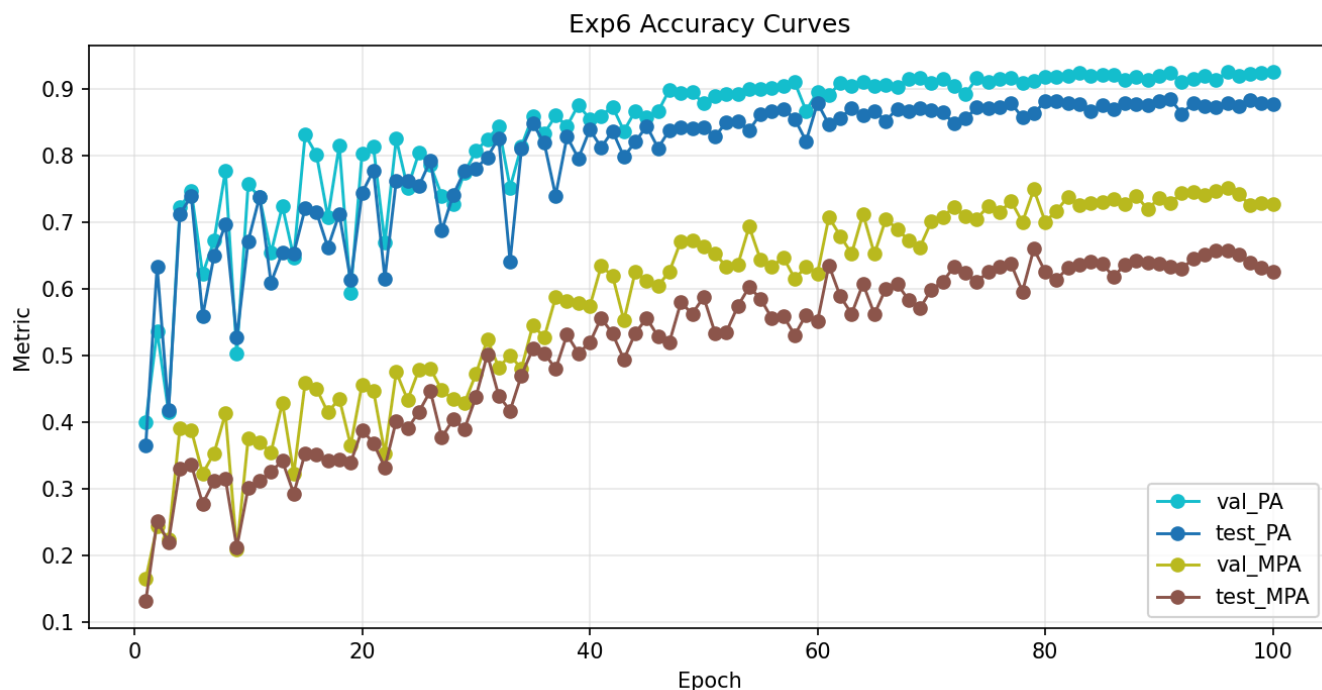
训练日志有三张曲线图，分别对应损失、mIoU 和 accuracy 的变化趋势：

Exp6 Loss Curves



Exp6 mIoU Curves

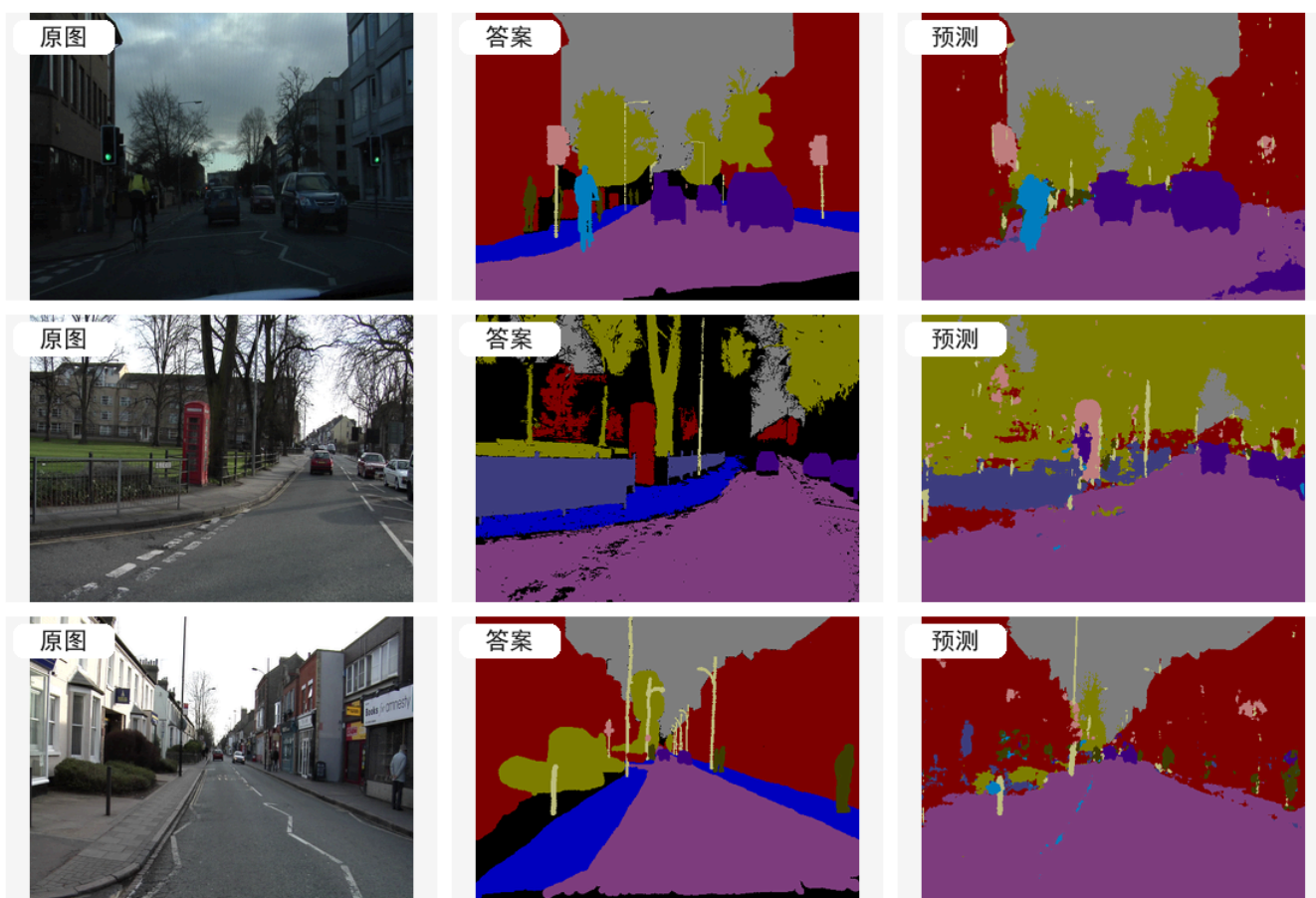




可以发现一开始训练速度较快，但是随着时间推移，损失下降和 mIoU 提升逐渐减缓，说明模型在主类上已接近收敛，而小目标类别仍然存在较大提升空间。

7.2 预测结果示例

下图展示了预测结果与原图、标签的对比，三组样例分别选自测试集中的 0001TP_008550、Seq05VD_f00000 和 Seq05VD_f01020。



从对比结果可以看出：

1. 原图与答案之间具有较清晰的场景对应关系，适合观察分割边界。
2. 预测结果对天空、道路和建筑等主类较稳定，整体轮廓也比较连贯。
3. 小目标与边界类仍有一定缺失和粘连，尤其是行人、杆件与路缘区域。

8. 结论与改进方向

8.1 结论

本实验已完成 CamVid 语义分割流程的工程闭环（训练/评估/预测/可视化）。在当前配置下，模型对主类表现较好，测试集达到 mIoU 0.5177；但在小目标类别与道路边界细分方面仍有明显短板。

8.2 后续改进

可能的优化方向：

1. 损失函数增强：CE + Dice 或 CE + Focal，提升小目标召回。
2. 结构增强：引入更强 backbone 或轻量级多尺度特征融合。
3. 训练策略：更长训练周期、类别重采样、困难样本挖掘。
4. 推理策略：多尺度与水平翻转 TTA。
5. 数据与标注：重点排查 Pavement 与 Road 的标签一致性。

9. 复现实验命令（提交目录）

快速过测（推荐）：

```
powershell -ExecutionPolicy Bypass -File src_to_submit/exp6/one_click_smoke.ps1
```

单独评估：

```
python src_to_submit/exp6/main.py --mode eval --resume_ckpt  
src/exp6/checkpoints/segnet_camvid.best.pth
```

单独预测：

```
python src_to_submit/exp6/main.py --mode predict --resume_ckpt  
src/exp6/checkpoints/segnet_camvid.best.pth --pred_dir  
src_to_submit/exp6/visualizations/predictions_quick
```